



Instituto Tecnológico Superior de Lerdo

Desarrollo de Aplicaciones para Dispositivos Móviles

Unidad 4

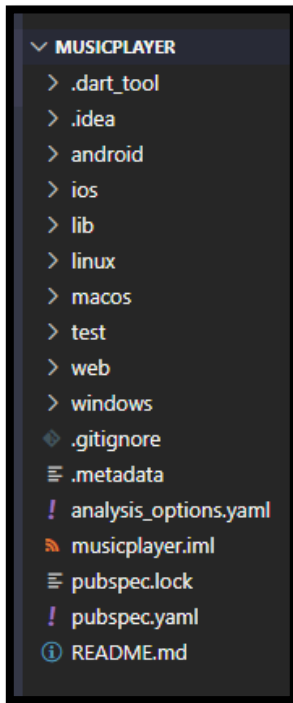
Actividad 4.1 - Creación de un Mini Reproductor de Música en Flutter

Alumna: Cristal Ruiz Lozano #222310571

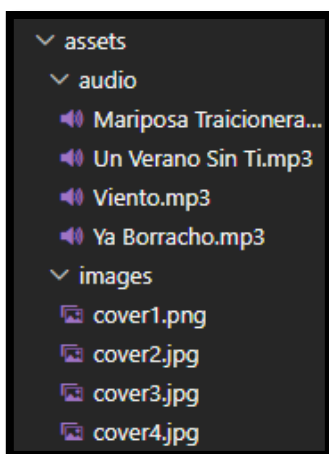
Docente: Jesús Salas Marín

28/05/2026

Para comenzar con el desarrollo de la práctica, se creó un nuevo proyecto Flutter utilizando la combinación de teclas **Ctrl + Shift + P** dentro de Visual Studio Code. Posteriormente se escribió la opción “**Flutter: New Project**” y se seleccionó el tipo “**Application**”. Después se eligió la carpeta donde se almacenaría el proyecto, generando automáticamente la estructura base necesaria para trabajar con Flutter y Dart.

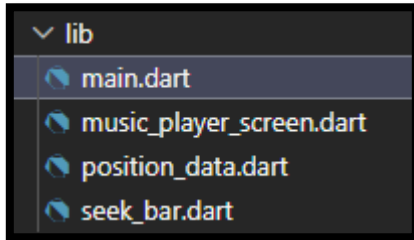


Posteriormente se creó una carpeta llamada **assets**, la cual contiene dos subcarpetas importantes llamadas **audio** e **images**. La carpeta audio almacena las canciones en formato MP3 utilizadas por el reproductor de música, mientras que images contiene las portadas o imágenes que serán mostradas dentro de la interfaz de la aplicación.



Dentro de la carpeta “**lib**” se encuentran los principales archivos utilizados para el desarrollo del reproductor de música:

- **Main.dart**
- **Music_player_screen.dart**
- **position_data.dart**
- **seek_bar.dart**



Comencemos con el archivo **main.dart**, el cual funciona como el punto de entrada principal de la aplicación Flutter. Este archivo se encarga de iniciar la aplicación y mostrar la pantalla principal del reproductor de música.

Primero, se importan las librerías necesarias. La librería **material.dart** proporciona los widgets y componentes visuales de Flutter basados en Material Design. También se importa el archivo **music_player_screen.dart**, el cual contiene la interfaz principal del reproductor de música.

```
import 'package:flutter/material.dart';
import 'music_player_screen.dart';
```

Posteriormente se encuentra la función **main()**, la cual es la primera función que se ejecuta al iniciar la aplicación. Dentro de ella se utiliza **runApp()**, encargado de cargar el widget principal llamado **MusicPlayerApp**.

```
void main() {
  runApp(const MusicPlayerApp());
}
```

Después se crea la clase **MusicPlayerApp**, la cual extiende de **StatelessWidget**. Esto significa que el widget no maneja estados dinámicos y únicamente se utiliza para definir la configuración principal de la aplicación.

```
class MusicPlayerApp extends StatelessWidget {
  const MusicPlayerApp({super.key});
```

Dentro del método **build()** se retorna un widget **MaterialApp**, el cual funciona como la estructura principal de la aplicación. Aquí se desactiva la etiqueta de depuración utilizando **debugShowCheckedModeBanner: false**, se asigna el nombre de la aplicación mediante **title** y se establece como pantalla principal el widget **MusicPlayerScreen()**.

```

@override
Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    title: 'Music Player',
    home: const MusicPlayerScreen(),
  ); // MaterialApp
}
}

```

Código del archivo **main.dart**:

```

// Importa los widgets principales de Flutter
import 'package:flutter/material.dart';

// Importa la pantalla principal del reproductor
import 'music_player_screen.dart';

// Función principal que inicia la aplicación
void main() {
  runApp(const MusicPlayerApp());
}

// Widget principal de la aplicación
class MusicPlayerApp extends StatelessWidget {
  const MusicPlayerApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      // Oculta la etiqueta de DEBUG
      debugShowCheckedModeBanner: false,

      // Nombre de la aplicación
      title: 'Music Player',

      // Pantalla principal que se mostrará al iniciar
      home: const MusicPlayerScreen(),
    );
  }
}

```

El archivo **position_data.dart** contiene una clase utilizada para almacenar la información relacionada con la reproducción del audio dentro de la aplicación. Su función principal es organizar los datos del tiempo de reproducción de las canciones, permitiendo controlar el progreso del audio en la barra de reproducción.

La clase **PositionData** maneja tres valores importantes: la posición actual de la canción (**position**), la parte del audio que ya fue cargada (**bufferedPosition**) y la duración total de la pista (**duration**). Estos datos son utilizados por el reproductor para actualizar visualmente la barra de progreso y mostrar correctamente el estado de reproducción del audio.

Además, este archivo ayuda a mantener el código más organizado, ya que centraliza la información relacionada con el tiempo de reproducción en una sola clase.

Código del archivo **position_data.dart**:

```
// Clase que almacena información sobre la reproducción del audio
class PositionData {

  // Tiempo actual de reproducción
  final Duration position;

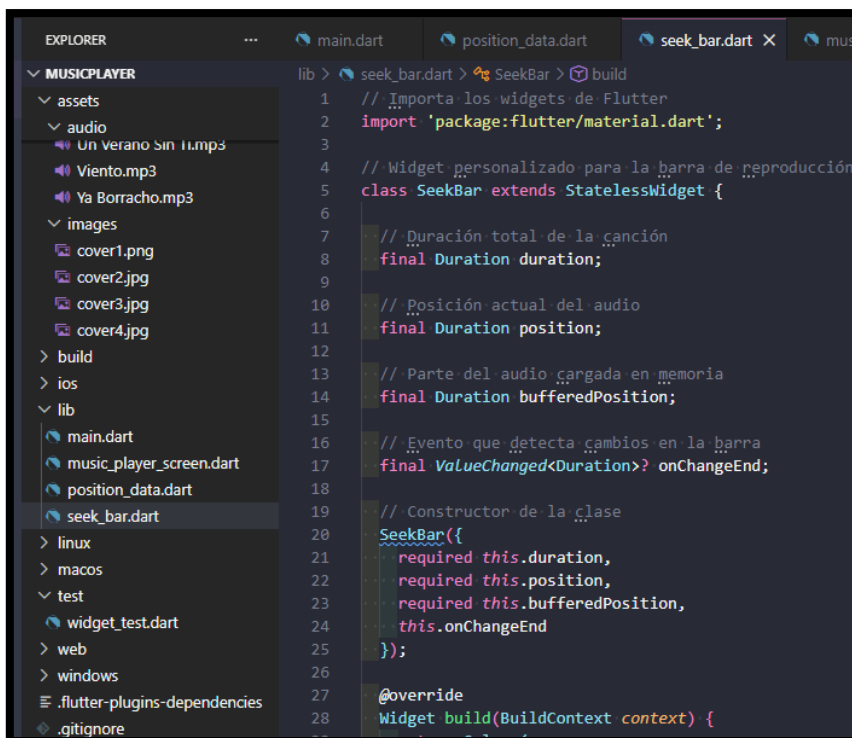
  // Parte del audio que ya fue cargada
  final Duration bufferedPosition;

  // Duración total de la canción
  final Duration duration;

  // Constructor de la clase
  PositionData(
    this.position,
    this.bufferedPosition,
    this.duration,
  );
}
```

El archivo “**seek_bar.dart**” contiene la implementación de una barra de progreso personalizada utilizada dentro del reproductor de música. Su función principal es mostrar el avance actual de la canción, permitir cambiar la posición de reproducción del audio y visualizar tanto el tiempo transcurrido como la duración total de la pista.

Este archivo utiliza el widget “**Slider**” de Flutter para crear una barra interactiva que permite mover manualmente la reproducción de la canción. Además, incluye un método encargado de convertir la duración del audio a un formato de minutos y segundos para facilitar su visualización dentro de la interfaz.



El archivo **music_player_screen.dart** contiene la pantalla principal del mini reproductor de música. En este archivo se implementa toda la lógica relacionada con la reproducción de audio, el control de canciones, las animaciones y el diseño visual de la interfaz.

Primero se importan las librerías necesarias para el funcionamiento del reproductor. Se utilizan librerías de Flutter para la interfaz gráfica, **just_audio** para la reproducción de audio, **rxdart** para manejar flujos de datos y archivos personalizados como **position_data.dart** y **seek_bar.dart** para controlar la barra de progreso y la información del audio.

Posteriormente se crea la clase **MusicPlayerScreen**, la cual representa la pantalla principal del reproductor. Esta clase utiliza **StatefulWidget**, ya que la interfaz necesita actualizarse constantemente dependiendo del estado de reproducción, la canción seleccionada y el progreso del audio.

Dentro de la clase principal se inicializan varios elementos importantes. Se crea un objeto **AudioPlayer** encargado de reproducir las canciones, un **AnimationController** utilizado para generar la animación giratoria de la portada del disco y una lista llamada **playlist**, donde se almacenan las canciones junto con su artista, archivo de audio e imagen correspondiente.

La función **initState()** se ejecuta al iniciar la pantalla y se encarga de inicializar el reproductor, configurar la lista de canciones y preparar las animaciones utilizadas dentro de la interfaz.

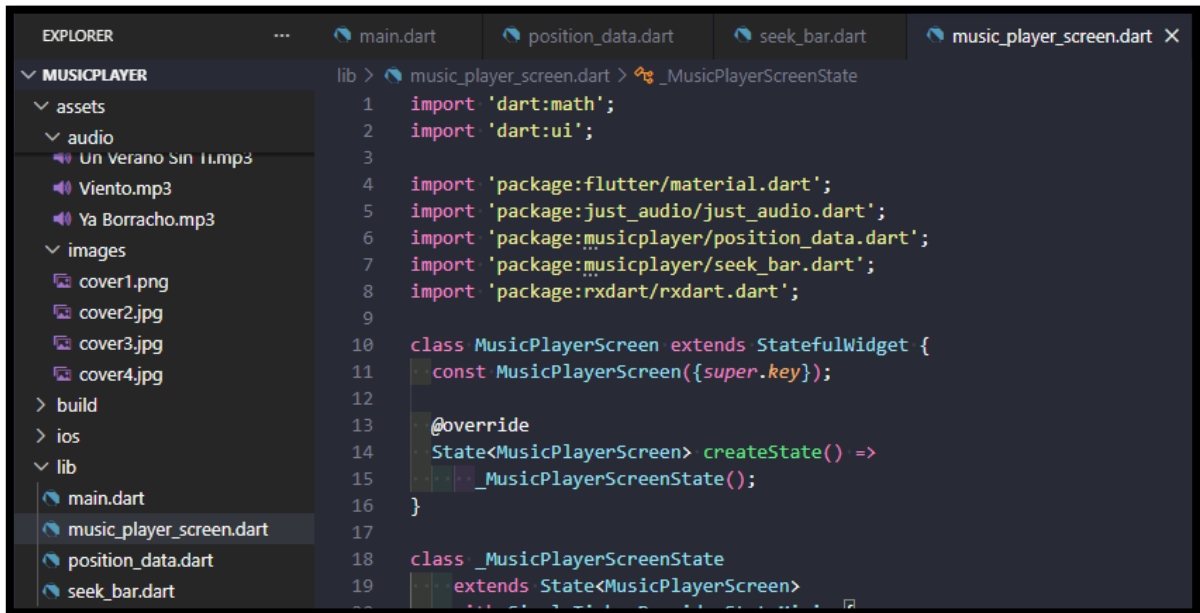
También se implementan funciones como **_playPause()**, **_nextSong()** y **_previousSong()**, las cuales permiten controlar la reproducción de música mediante los botones de Play, Pause, siguiente canción y canción anterior.

Además, se utiliza un Stream llamado **_positionDataStream** para actualizar en tiempo real la barra de progreso del audio y mostrar correctamente la duración y posición actual de la canción.

En el método **build()** se construye toda la interfaz visual del reproductor. Se diseña un fondo con degradados oscuros y un estilo moderno tipo vinilo vintage. La portada de la canción se muestra dentro de un disco circular animado que gira mientras la música se reproduce. También se visualiza el nombre de la canción, el artista, la barra de progreso y los controles de reproducción.

Finalmente, el método **dispose()** se encarga de liberar la memoria utilizada por el reproductor de audio y las animaciones al cerrar la aplicación, evitando problemas de rendimiento o consumo innecesario de recursos.

En general, este archivo representa el núcleo principal del mini reproductor de música, ya que integra tanto la lógica de reproducción como el diseño visual y las animaciones de la aplicación.



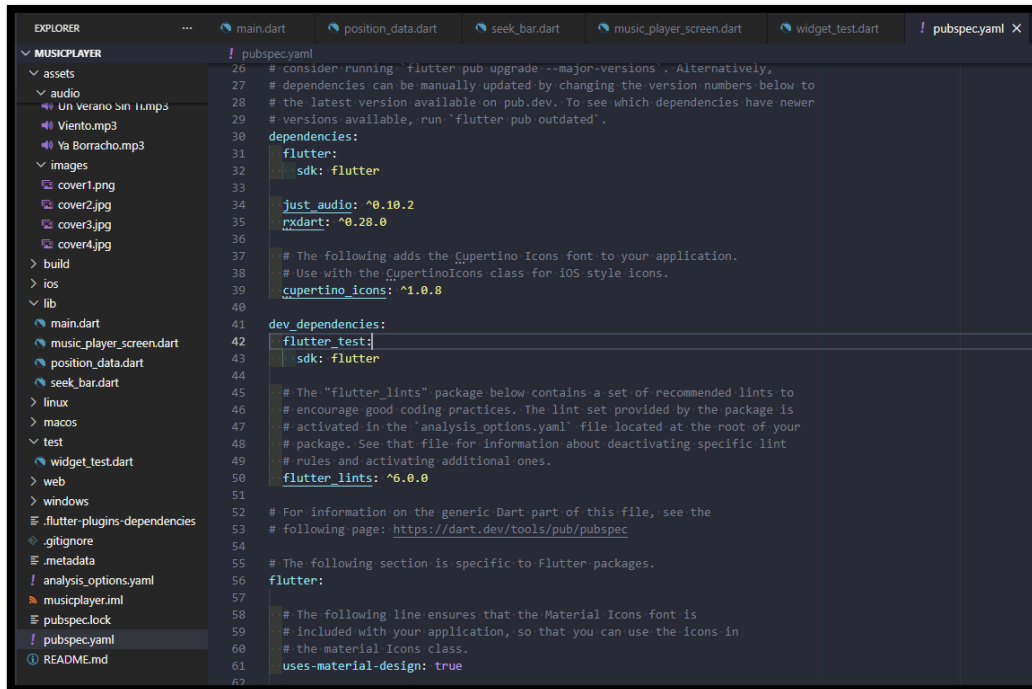
```
lib > music_player_screen.dart > _MusicPlayerScreenState
1  import 'dart:math';
2  import 'dart:ui';
3
4  import 'package:flutter/material.dart';
5  import 'package:just_audio/just_audio.dart';
6  import 'package:musicplayer/position_data.dart';
7  import 'package:musicplayer/seek_bar.dart';
8  import 'package:rxdart/rxdart.dart';
9
10 class MusicPlayerScreen extends StatefulWidget {
11   const MusicPlayerScreen({super.key});
12
13   @override
14   State<MusicPlayerScreen> createState() =>
15     _MusicPlayerScreenState();
16 }
17
18 class _MusicPlayerScreenState
19   extends State<MusicPlayerScreen>
```

El archivo **pubspec.yaml** es uno de los archivos más importantes dentro de un proyecto Flutter, ya que se utiliza para configurar la aplicación y administrar las dependencias necesarias para su funcionamiento. En este archivo se definen datos generales del proyecto, como el nombre de la aplicación, la descripción, la versión y la versión del SDK de Dart utilizada.

Además, en este archivo se agregaron las librerías externas necesarias para el desarrollo del mini reproductor de música, como **just_audio**, utilizada para reproducir archivos de audio, y **rxdart**, empleada para manejar flujos de datos y actualizar la información de reproducción en tiempo real.

También se configuró la sección **assets**, donde se registraron las carpetas **audio** e **images**. Esto permitió que Flutter reconociera correctamente las canciones y las imágenes utilizadas dentro de la aplicación.

Finalmente, al guardar los cambios realizados en el archivo **pubspec.yaml**, Flutter descargó automáticamente todas las dependencias necesarias para el funcionamiento del proyecto, preparando el entorno para ejecutar correctamente la aplicación.



Ejecución

Una vez finalizada la programación y configuración del proyecto, se utilizó la terminal integrada de Visual Studio Code para ejecutar la aplicación.

Primero se ejecutó el siguiente comando: **flutter pub get**

Este comando permitió descargar automáticamente todas las dependencias necesarias del proyecto, como las librerías `just_audio` y `rxdart`.

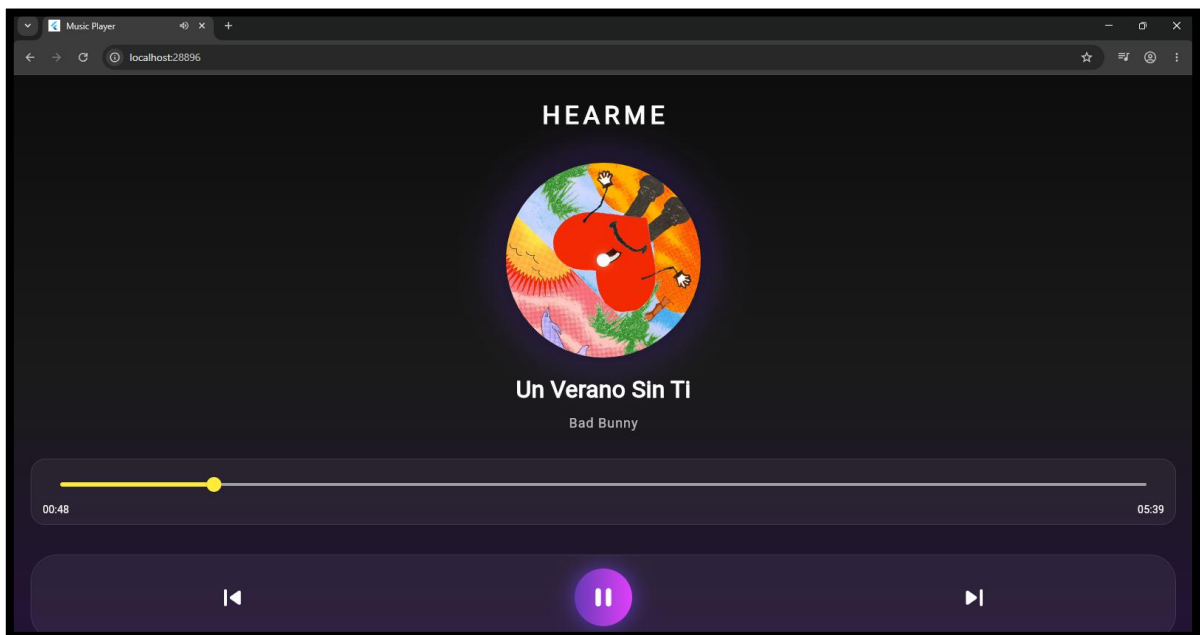
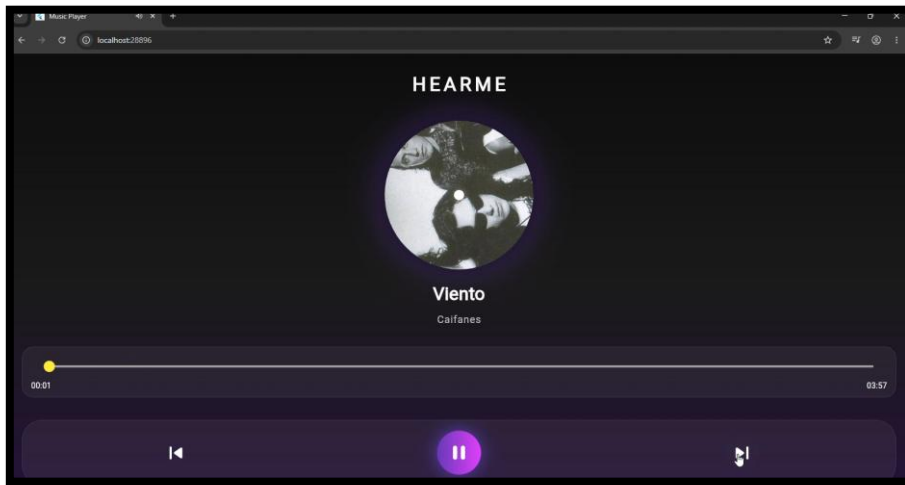
Posteriormente se ejecutó la aplicación utilizando el comando: **flutter run**

Después de compilar el proyecto, Flutter abrió automáticamente la aplicación en el navegador seleccionado, permitiendo probar el funcionamiento del mini reproductor de música y verificar que la reproducción de audio, animaciones y controles funcionaran correctamente.

Resultados

Como resultado de la práctica se desarrolló un mini reproductor de música funcional utilizando Flutter y Dart. La aplicación permite reproducir canciones en formato MP3, pausar la música, avanzar o retroceder entre pistas y visualizar el progreso de reproducción mediante una barra interactiva.

Además, se personalizó la interfaz gráfica con un diseño moderno y estilo vintage, incorporando animaciones, imágenes de portada para cada canción y efectos visuales dinámicos.



Preguntas de reflexión

1. ¿Qué retos encontraste al seguir las instrucciones del video?

Uno de los principales retos fue configurar correctamente las librerías y los archivos multimedia dentro de la carpeta assets. También surgieron algunos problemas relacionados con la reproducción de canciones y el cambio entre pistas, los cuales tuvieron que corregirse realizando pruebas y ajustes en el código.

2. ¿Qué diferencia notaste entre una aplicación que simula una interfaz y una que realmente reproduce audio?

Una aplicación que solo simula una interfaz únicamente muestra elementos visuales sin ejecutar funciones reales. En cambio, una aplicación que realmente reproduce audio necesita manejar procesos como la carga de canciones, sincronización de la barra de progreso, control de reproducción y administración de estados en tiempo real.

3. ¿Qué widgets consideras más importantes o esenciales para construir una interfaz de un reproductor en Flutter?

Algunos de los widgets más importantes fueron Scaffold, Container, Column, Row, Text, IconButton, Slider y AnimatedBuilder, ya que permitieron construir la interfaz, organizar los elementos visuales, controlar la reproducción de música y agregar animaciones al reproductor.